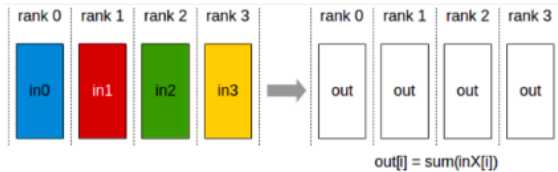


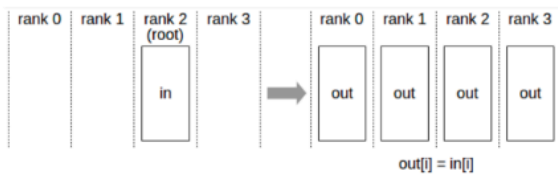
# parallelism basics

## But first.. Some basics about collective communication

### All reduce



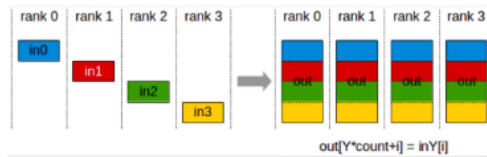
### Broadcast



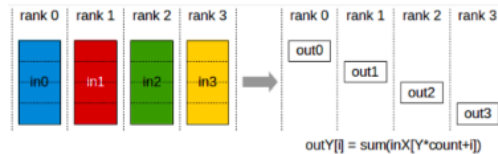
### Reduce



### All Gather



### Reduce Scatter



这张图展示的是分布式计算（Parallel/Distributed Computing）中核心的“集合通信”（Collective Communication）模式。这些模式主要用于多张显卡（GPU）或多台机器之间如何高效地交换数据和同步结果。

为了让你更直观地理解，我们可以把这4个 Rank（Rank 0 ~ Rank 3）想象成4个正在一起做数学题的学生。

以下是形象化的解释：

## 1. Broadcast（广播）

- 场景：老师发卷子。
- 动作：
  - Rank 2（Root，根节点）手里有一份唯一的标准答案（数据 in）。
  - 他把这份答案复印了 3 份，分发给 Rank 0、Rank 1 和 Rank 3。
- 结果：现在 4 个学生手里都有了一模一样的那份答案。
- 一句话总结：一人有，分给全员。

## 2. Reduce（归约）

- **场景：**小组长收班费并计算总额。
  - **动作：**
    - 4 个学生 (Rank 0-3) 手里各自有一笔钱 (`in0`, `in1`, `in2`, `in3`)。
    - 大家把钱都交给 **Rank 2 (Root, 组长)**。
    - Rank 2 把这些钱加在一起 (Sum 操作)。
  - **结果：**只有 Rank 2 知道总金额是多少，其他人手里空了或者不知道总数。
  - **一句话总结：**全员上交，一人汇总。
- 

### 3. All Reduce (全归约)

- **场景：**大家一起对账，确保每个人都知道总账。
  - **动作：**
    - 这是分布式训练中最常用的操作。
    - 4 个学生手里各自有一笔钱。
    - **第一步：**大家先把钱汇总起来算出总数 (就像 Reduce)。
    - **第二步：**把这个总数告诉每一个人 (就像 Broadcast)。
  - **结果：**4 个学生手里都拿到了所有人的钱的总和。
  - **一句话总结：**人人贡献，人人知道总结果。
- 

### 4. All Gather (全收集)

- **场景：**每人带一个菜的“百家宴” (Potluck)。
  - **动作：**
    - Rank 0 带了米饭，Rank 1 带了肉，Rank 2 带了菜，Rank 3 带了汤。
    - 大家把菜放在一个大转盘上，转一圈。
  - **结果：**最后每个人的盘子里都有【米饭、肉、菜、汤】这四样东西的组合。注意，这里没有做加法计算，只是把数据拼在了一起。
  - **一句话总结：**每人出一点，最后每人拥有一份完整的拼盘。
- 

### 5. Reduce Scatter (归约散播)

- **场景：**分工计算总分。
- **动作：**
  - 假设有 4 个科目的考试分数需要统计。如果让一个人算太累了。

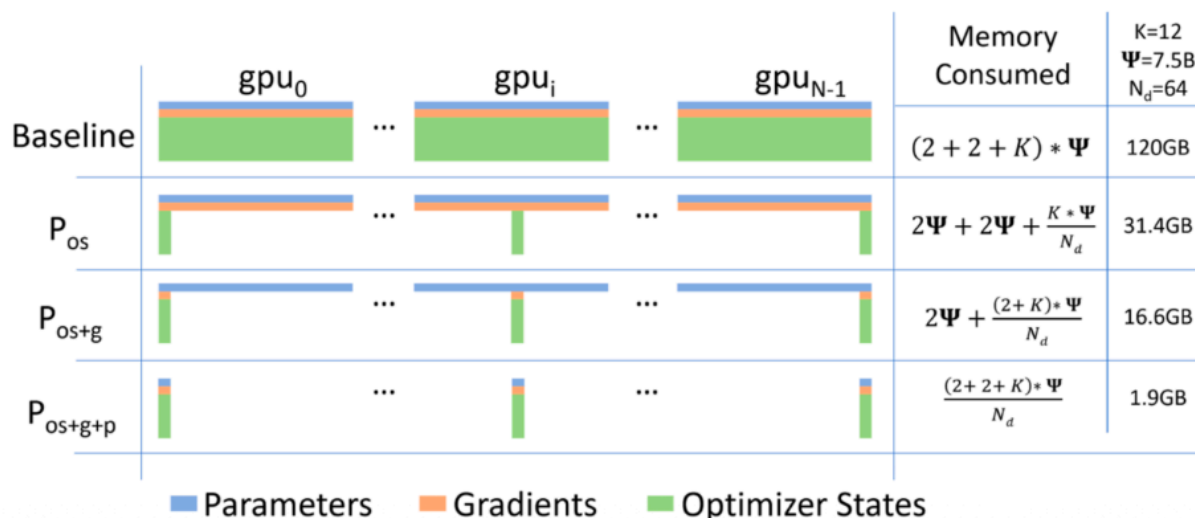
- 大家手里都有这 4 个科目的部分分数。
- 分工：Rank 0 负责算“语文”的总分，Rank 1 负责算“数学”的总分，Rank 2 负责“英语”，Rank 3 负责“物理”。
- 大家把各自手里的数据，按照科目扔给对应的负责人，负责人进行累加。
- 结果：
  - Rank 0 手里只有语文的总分。
  - Rank 1 手里只有数学的总分。
  - （以此类推...）
  - 整体来看，总分算出来了，但是分散在不同的人手里。
- 一句话总结：按任务分工汇总，每人只拿自己负责的那部分结果。

## 总结图谱

| 模式                    | 谁有原始数据？     | 谁得到结果？        | 做了什么？       | 形象比喻  |
|-----------------------|-------------|---------------|-------------|-------|
| <b>Broadcast</b>      | 只有一人 (Root) | 所有人           | 复制分发        | 大喇叭喊话 |
| <b>Reduce</b>         | 所有人         | 只有一人 (Root)   | 汇总计算 (Sum)  | 组长收钱  |
| <b>All Reduce</b>     | 所有人         | 所有人           | 汇总计算 (Sum)  | 全员对账  |
| <b>All Gather</b>     | 所有人 (每人一部分) | 所有人 (每人全套)    | 拼接 (Concat) | 凑份子聚餐 |
| <b>Reduce Scatter</b> | 所有人         | 所有人 (每人一部分结果) | 汇总并分片       | 分工算账  |

# ZeRO – solving the memory overhead issue of DP

**Core idea:** split up the expensive parts (state) and use the reduce-scatter equivalence.



这张图展示的是 **ZeRO (Zero Redundancy Optimizer)** 技术的核心原理。这是微软 DeepSpeed 框架中提出的一种技术，专门用来解决训练超大模型时显存不够用的问题。

简单来说，它的核心思想是：既然多张显卡在一起干活，为什么要每张卡都存一份一模一样的数据呢？不如把数据切分（Partition/Shard），每张卡只存一部分。

我们可以把这三层优化（Stage 1, 2, 3）想象成给一群学生（GPU）减负的过程。

## 1. 图例解释

- **蓝色 (Parameters):** 模型参数（比如神经网络的权重）。
- **橙色 (Gradients):** 梯度（训练过程中计算出的调整方向）。
- **绿色 (Optimizer States):** 优化器状态（比如 Adam 优化器需要的动量、方差等，这部分其实占内存最大）。
- **横轴:** 不同的 GPU ( $gpu_0$  到  $gpu_{N-1}$ )。
- **纵轴:** 不同的优化阶段。

## 2. 详细解读四个阶段

### Baseline（基线：传统的各种并行）

- **状态:** 全员负重。
- **现象:** 你看  $gpu_0$ 、 $gpu_i$  每一张卡上，蓝色、橙色、绿色条都是满的。

- **含义：**这是传统的数据并行（Data Parallelism）。每张显卡都必须完整地存下一份模型、一份梯度、一份优化器状态。
- **后果：**显存占用极大（120GB）。如果模型太大，单张卡根本放不下，训练直接报错（OOM）。

### $P_{os}$ (ZeRO Stage 1): 切分优化器状态

- **状态：**把最重的包袱分了。
- **做法：**绿色条（Optimizer States）被切碎了，每张卡只存一小段绿色。蓝色和橙色还是满的。
- **原理：**在 Adam 优化器中，优化器状态占用的显存通常是模型参数的 3 倍以上（也就是那个  $K = 12$ ）。ZeRO Stage 1 让每张卡只负责更新模型的一小部分参数，所以它只需要存那一小部分的优化器状态。
- **效果：**显存占用瞬间从 120GB 降到了 31.4GB。这是性价比最高的一步。

### $P_{os+g}$ (ZeRO Stage 2): 切分梯度

- **状态：**把草稿纸也分了。
- **做法：**橙色条（Gradients）也被切碎了。
- **原理：**既然每张卡只负责更新那一小部分参数，那它也只需要保留那一小部分的梯度。其他部分的梯度算完后，通过通信（Reduce-Scatter）交给负责的卡，自己就不存了。
- **效果：**显存进一步降到 16.6GB。

### $P_{os+g+p}$ (ZeRO Stage 3): 切分模型参数

- **状态：**把课本也分了（极限瘦身）。
- **做法：**蓝色条（Parameters）也被切碎了。现在每张卡上，蓝、橙、绿都只有极小的一条。
- **原理：**每张卡只存属于它负责的那一小部分模型参数。
  - *问题来了：那我要计算的时候需要完整的模型怎么办？*
  - *解决：*需要用到哪部分参数，就临时通过通信（Broadcast/AllGather）从别的卡那里借过来，算完马上扔掉，不占显存。
- **效果：**显存占用降到了惊人的 1.9GB。这意味着你可以用同样的硬件训练比原来大几十倍的模型。

---

## 3. 形象化比喻

假设有 64 个学生（GPU）要一起背诵并抄写一本巨厚的《百科全书》（大模型）。

- **Baseline：**每人发一本完整的书（参数），每人发一本完整的草稿本（梯度），每人还要背着一个巨大的书包（优化器状态）。**结果：**书桌（显存）直接被压塌了。

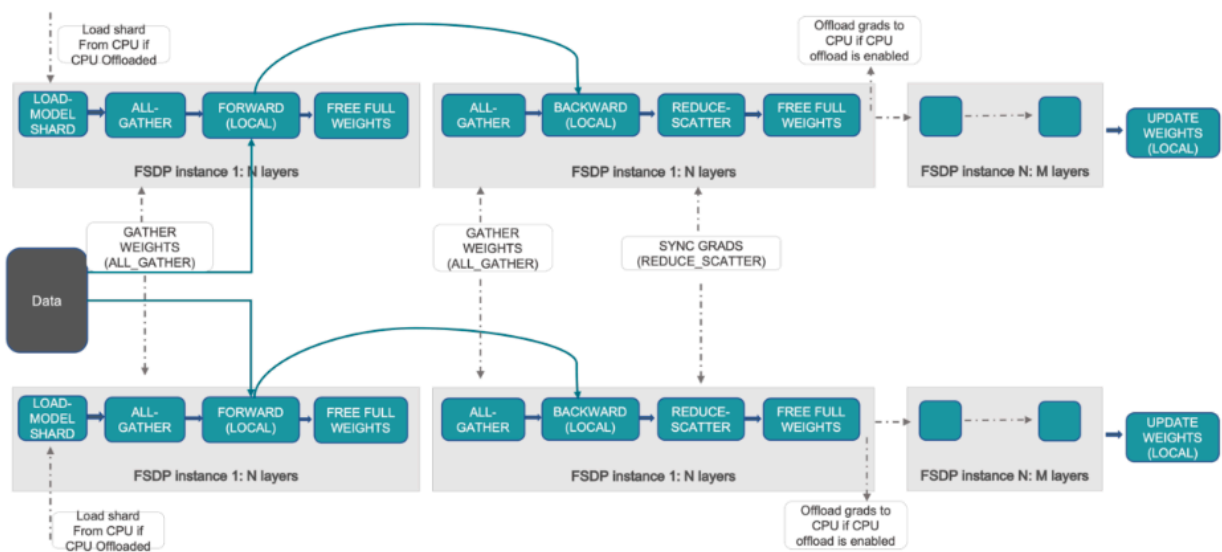
- **Stage 1 ( $P_{os}$ )**: 书和草稿本还是每人一本，但巨大的书包里的东西大家分摊。每个人只背 1/64 的物资。**结果：书桌轻松了很多。**
- **Stage 2 ( $P_{os+g}$ )**: 草稿本也分摊了。每个人只负责修整全书第 10 页的错别字，所以你只需要第 10 页的草稿纸。**结果：书桌更空了。**
- **Stage 3 ( $P_{os+g+p}$ )**: 连书都撕开了！每个人手里只有几页书。
  - 怎么读？当你需要读第 1 页时，拿第 1 页的那个同学大声朗读（Broadcast），大家听完记在脑子里算一下，算完就忘掉。
  - 结果：书桌上几乎是空的，可以用来放更大的新书（训练更大的模型）。

## 总结

这张图完美展示了 "用通信换显存" 的思想。

- 越往下（Stage 1 -> 3），显存占用越小，能训练的模型越大。
- 代价是通信量会变大（因为参数不再是本地现成的，需要不停地问别人要）。但在现代高带宽集群下，这个交换是非常值得的。

## ZeRO stage 3 (aka FSDP) how it works (baby version)



Communication cost – 2 all gather (#param), 1 reduce-scatter (#param).

[https://pytorch.org/tutorials/intermediate/FSDP\\_tutorial.html](https://pytorch.org/tutorials/intermediate/FSDP_tutorial.html)

这张图展示的是 **ZeRO Stage 3**（在 PyTorch 中通常被称为 **FSDP: Fully Sharded Data Parallel**）的工作原理。

如果说 Stage 1 是“分摊书包（优化器）”，Stage 2 是“分摊草稿纸（梯度）”，那么 **Stage 3** 就是最极致的“把课本（模型参数）也撕开分摊了”。

## 1. 核心流程解读 (How it works)

这张图展示了两个并行的 GPU（上下两行）是如何协同工作的。核心逻辑是：“平时手里没参数，要算的时候临时借，算完立马扔。”

### 前向传播 (Forward Pass)

1. **Load Model Shard**: 每个人手里只有一小块模型碎片 (Shard)。
2. **All-Gather (关键动作)**:
  - 我要算这一层了，但我手里的参数不全啊。
  - **动作**：所有人把各自手里的那一小块参数广播出来。
  - **结果**：瞬间，每张卡都在显存里拼凑出了这一层完整的参数 (Full Weights)。
3. **Forward (Local)**: 参数全了，开始做前向计算。
4. **Free Full Weights (关键动作)**:
  - 这一层算完了？好，立刻把刚才借来的完整参数删掉！
  - 只保留自己原本负责的那一小块碎片。
  - **目的**：腾出显存给下一层用。

### 反向传播 (Backward Pass)

1. **All-Gather**:
  - 又要算梯度了，参数又不全了。
  - **动作**：再做一次全收集，把这一层的完整参数拼凑出来。
2. **Backward (Local)**: 计算这一层的梯度。
3. **Reduce-Scatter**:
  - 算出了完整的梯度，但我不需要存这么多。
  - **动作**：大家把梯度汇总，然后切分。我只拿回我负责的那一小块梯度。
4. **Free Full Weights**:
  - 算完了，再次把借来的完整参数删掉。

### 参数更新 (Update)

- **Update Weights**: 最后，每个人用自己手里那一小块梯度，更新自己手里那一小块模型碎片。

---

## 2. Stage 1, 2, 3 的终极对比

我们可以用“组装高达模型”来比喻这三个阶段。假设我们要组装一个巨大的高达（大模型），有 64 个人（GPU）一起干活。

## Baseline (传统数据并行)

- 做法：每个人桌子上都放一整套高达零件、一整套说明书、一整套工具箱。
- 问题：桌子（显存）根本放不下，直接被压垮。

## Stage 1 (优化器状态切分)

- 做法：
  - 零件（参数）：每人桌上还是全套。
  - 说明书（梯度）：每人桌上还是全套。
  - 工具箱（优化器状态）：大家分摊了！每个人只拿一把螺丝刀或一把钳子。
- 效果：桌子腾出了一些空间，能装得下了。

## Stage 2 (梯度切分)

- 做法：
  - 零件（参数）：每人桌上还是全套。
  - 说明书（梯度）：大家分摊了！我只负责看第 10 页说明书，看完就把这页扔了，不占地儿。
  - 工具箱：分摊了。
- 效果：桌子空间更大了。

## Stage 3 / FSDP (参数切分 - 本图内容)

- 做法：
  - 零件（参数）：大家分摊了！我桌上只有高达的“左手大拇指”。
  - 怎么拼？当我们需要拼“左手”时，大家把各自手里的手指头零件扔到中间桌子上，临时拼好左手，操作一下，然后马上拆散拿回各自零件。
  - 说明书：分摊了。
  - 工具箱：分摊了。
- 效果：
  - 显存占用极低：桌子上几乎是空的。
  - 通信开销变大：因为要不停地“扔出来拼好、拆散拿回去”（All-Gather 和 Free），网络通信非常频繁。

## 3. 总结图中的通信成本 (Communication Cost)

图中最下方写着：

```
Communication cost - 2 all gather (#param), 1 reduce-scatter (#param)
```

这意味着在一次完整的迭代（Forward + Backward）中，每个参数要被传输 3 次：



1. 前向传播时: All-Gather 一次 (为了凑齐参数算 Forward)。
2. 反向传播时: All-Gather 一次 (为了凑齐参数算 Backward)。
3. 反向传播结束时: Reduce-Scatter 一次 (为了同步梯度并拆分)。

结论: ZeRO Stage 3 是用“网络带宽”换“显存空间”的极致方案。如果你的显卡显存很小, 但网速很快 (比如有 NVLink), 用这个模式最划算。

这份总结旨在帮助你快速制作一份关于 分布式训练与 ZeRO/FSDP 优化 的技术分享 PPT。

这份 Slide 大纲按照逻辑递进, 从“基础概念”到“核心痛点”, 再到“解决方案”和“深度原理解析”。

---

## 分享主题: 大模型训练显存优化 —— 从集合通信到 ZeRO/FSDP

### Slide 1: 基础底座 —— 集合通信 (Collective Communication)

核心概念: 多卡之间如何高效交换数据? 这是分布式训练的基石。

- **Broadcast (广播):** Root 节点把数据复制给所有人。(一人喊话, 全员听到)
- **Reduce (归约):** 所有人把数据交给 Root 节点汇总。(全员上交, 班长统计)
- **All-Reduce (全归约):** 最常用的操作。所有人贡献数据, 最后所有人都得到汇总结果。(全员对账, 信息同步)
- **All-Gather (全收集):** 每个人出一部分数据, 最后每个人都得到完整的数据拼盘。(凑份子聚餐)
- **Reduce-Scatter (归约散播):** 汇总结果后, 每个人只拿回自己负责的那一部分结果。(分工记账)

---

### Slide 2: 核心痛点 —— 显存墙 (The Memory Wall)

问题背景: 为什么普通的数据并行 (DP) 训不动大模型?

- **Baseline (传统 DP):**
  - 每张 GPU 都要存 3 份完整数据:
    1.  **Parameters (模型参数)**
    2.  **Gradients (梯度)**
    3.  **Optimizer States (优化器状态)** —— 这是最大的显存杀手!
  - 后果: 显存冗余严重, 模型稍大即 OOM (Out of Memory)。
- **ZeRO 的核心思想:**

- 去除冗余 (Zero Redundancy)。既然大家一起干活，没必要每人背一份全套物资。
- 切分 (Sharding)：把数据切碎，分散存储在不同的 GPU 上。

## Slide 3: 解决方案 —— ZeRO 的三个阶段 (The 3 Stages)

进阶之路：如何一步步把显存“榨干”？

| 阶段      | 优化策略 (切分什么?)               | 显存占用                                                                                             | 通信开销                                                                                            | 形象比喻                 |
|---------|----------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|----------------------|
| Stage 1 | 切分优化器状态 (Optimizer States) |  显著下降 (约节省 75%) |  低 (无额外通信)     | 行李分摊：大家分摊最重的背包。      |
| Stage 2 | + 切分梯度 (Gradients)         |  进一步下降 (再减半)    |  低 (利用反向传播间隙)  | 草稿纸分摊：算完一页撕一页，不存整本。  |
| Stage 3 | + 切分模型参数 (Parameters)      |  极致下降 (线性减少)    |  高 (增加 50% 通信) | 课本分摊：平时手里没书，要读哪页临时借。 |

## Slide 4: 深度解析 —— ZeRO Stage 2 workflow

核心机制：梯度是流动的，用完即弃。

1. **Backward (反向传播)**: 逐层计算梯度。
2. **Immediate Reduce (即时归约)**: 算完一层的梯度，立刻发给对应的负责人 (Reduce)。
3. **Free Memory (释放显存)**: 发送完毕后，立刻删除本地的梯度数据。
4. **Update (更新)**: 负责人用收到的一小部分梯度更新参数。
5. **All-Gather**: 更新完后，广播最新的参数给所有人。

## Slide 5: 终极形态 —— ZeRO Stage 3 (FSDP) workflow

核心机制：以时间换空间，以通信换显存。

- 关键动作: All-Gather -> Compute -> Free
- 前向传播 (Forward):
  - 需要算某一层时，通过 **All-Gather** 从别的卡借来完整参数。
  - 算完后，立刻释放 (**Free**) 完整参数，只留自己的碎片。
- 反向传播 (Backward):

- 再次 **All-Gather** 借来参数算梯度。
  - 算完梯度后，通过 **Reduce-Scatter** 汇总并切分梯度。
  - 再次释放完整参数。
  - 代价: 通信量是普通数据并行的 1.5 倍 (2次 All-Gather + 1次 Reduce-Scatter)。
- 

## Slide 6: 总结与选型建议 (Takeaway)

- 显存足够时: 使用 **DDP (Distributed Data Parallel)**，速度最快。
  - 显存有点紧: 开启 **ZeRO Stage 1** 或 **Stage 2**。性价比最高，几乎不影响速度，显存省一半以上。
  - 显存完全不够 (超大模型): 开启 **ZeRO Stage 3 (FSDP)**。
    - 虽然通信变慢了，但它是“能训”与“不能训”的区别。
    - *Tip:* 配合 CPU Offload 可以训练比显存大几十倍的模型。
- 

## 💡 演讲者备注 (Speaker Notes)

- 如果是讲给**算法工程师**: 重点强调 Stage 2 是目前最通用的配置 (HuggingFace Trainer 默认常开)。
- 如果是讲给**系统架构师**: 重点强调 Collective Communication 的带宽瓶颈，以及 Stage 3 对网络互联 (NVLink/InfiniBand) 的高要求。
- **归约 (Reduce)**
  - **功能**: 所有进程的数据进行某种运算 (如求和)，结果发送给一个 Root 进程。
  - **场景**: 梯度汇总。
  - **比喻**: 小组长收班费并计算总额。
- **全归约 (All-Reduce)**
  - **功能**: 所有进程的数据进行某种运算，并将结果发送给所有进程。
  - **场景**: 数据并行中同步梯度。
  - **比喻**: 每个人贡献数据，最后每个人都知道总结果。
- **全收集 (All-Gather)**
  - **功能**: 每个进程贡献一部分数据，最终所有进程都得到完整的数据集合。
  - **场景**: ZeRO Stage 3 中收集模型参数。
  - **比喻**: 百家宴，每人带一道菜，最后每人都能吃到所有菜。
- **归约散播 (Reduce-Scatter)**
  - **功能**: 所有进程的数据进行归约运算，然后将结果切片，分发给不同的进程。
  - **场景**: ZeRO Stage 2/3 中分发分片梯度。

- 比喻：分工计算总分，每人只拿自己负责的那部分结果。

#### 视觉元素建议：

- 使用动画或示意图展示每种通信模式的数据流向。
  - 可以参考第一张图的示意。
- 

## Slide 5: 显存墙：传统数据并行的瓶颈

标题：传统数据并行 (DP) 的显存困境

#### 核心内容：

在传统数据并行模式下，为了保证计算的独立性，每张 GPU 都需要存储模型的完整副本。

- **GPU 0, GPU 1, ..., GPU N-1**
  - **Parameters (模型参数)**: 完整副本
  - **Gradients (梯度)**: 完整副本
  - **Optimizer States (优化器状态)**: 完整副本 (通常是参数的 2-12 倍)
- **显存占用计算 (示例)**:
  - 假设模型参数量为  $\Psi$  (7.5B 参数)
  - 优化器状态因子  $K$  (如 Adam 需要 12 字节/参数)
  - **总显存 =  $(2 + 2 + K) \Psi = (2 + 2 + 12) 7.5B = 120 GB$**
  - **问题**：单张 A100 只有 80GB 显存！

#### 视觉元素建议：

- 使用原图中的 "Baseline" 部分，清晰展示每张 GPU 上的 Parameters, Gradients, Optimizer States 都是满的。
  - 突出显示 "120GB" 和 "OOM!" 警告。
- 

## Slide 6: ZeRO 优化器：零冗余的创新思想

标题：ZeRO (Zero Redundancy Optimizer)：打破显存壁垒

#### 核心内容：

ZeRO 的核心思想是**消除显存冗余**。它通过将模型训练状态 (Parameters, Gradients, Optimizer States) 进行分片 (**Sharding**)，使得每张 GPU 只存储这些状态的一部分，从而显著降低单卡显存占用。

- 核心理念：
  - “用通信换显存”：通过增加 GPU 之间的通信（数据传输），来减少每张 GPU 上的本地存储需求。
  - “按需加载”：只有在需要时才将完整数据组装起来，用完即释放。
- ZeRO 的三个阶段：
  - **Stage 1**: 分片优化器状态 (Optimizer States)
  - **Stage 2**: 分片梯度 (Gradients)
  - **Stage 3**: 分片模型参数 (Parameters)

#### 视觉元素建议：

- 一张图，显示一个完整的模型被切分成多个小块，分散到不同的 GPU 上。
  - “ZeRO”字样可以有“零”或“消除”的视觉效果。
- 

## Slide 7: ZeRO Stage 1: 优化器状态分片 ( $P_{os}$ )

标题：ZeRO Stage 1: 分摊最重的“行李”

#### 核心内容：

ZeRO Stage 1 (简称  $P_{os}$ ) 专注于分片优化器状态 (**Optimizer States**)。这是显存占用最大的一部分。

- 策略：
  - 每张 GPU 只存储自己负责更新的参数对应的优化器状态。
  - 模型参数和梯度仍然在每张 GPU 上完整复制。
- 显存占用计算 (示例):
  - 总显存 =  $(2 \Psi + 2 \Psi + K * \Psi / N_d) = 31.4 \text{ GB}$  (在  $N_d=64$  时)
- 效果：
  - 显著降低显存占用，通常可节省 75% 左右。
  - 通信开销几乎没有额外增加。
- 比喻：每个人还是拿着完整的书和草稿本，但最重的工具箱大家分摊了。

#### 视觉元素建议：

- 使用原图中的 " $P_{os}$ " 部分，突出显示绿色条 (Optimizer States) 被切分，而蓝色 (Parameters) 和橙色 (Gradients) 仍然完整。
  - 显示显存从 120GB 降到 31.4GB 的对比。
-

## Slide 8: ZeRO Stage 2: 梯度分片 ( $P_{os+g}$ )

标题：ZeRO Stage 2: 连“草稿纸”也分摊

核心内容：

ZeRO Stage 2 (简称  $P_{os+g}$ ) 在 Stage 1 的基础上，进一步分片**梯度 (Gradients)**。

- 策略：
  - 优化器状态 已分片。
  - 梯度 也被分片：每张 GPU 只存储自己负责更新的参数对应的梯度。
  - 模型参数 仍然在每张 GPU 上完整复制。
- 工作流 (关键):
  1. 反向传播时：每层梯度计算完成后，立即通过 **Reduce-Scatter** 操作将梯度汇总并分发给负责更新对应参数的 GPU。
  2. 发送后，本地的梯度数据立即释放。
  3. “用完即扔”：显存中不再保留完整的梯度副本。
- 显存占用计算 (示例):
  - 总显存 =  $(2 \Psi + 2 \Psi / N_d + K * \Psi / N_d) = 16.6 \text{ GB}$
- 效果：
  - 显存占用进一步显著降低，通常在 Stage 1 基础上再节省 50%。
  - 通信开销略有增加，但通常可以被高效的通信库隐藏。
- 比喻：每个人只有自己负责部分的草稿纸，算完就交给组长，自己立马扔掉。

视觉元素建议：

- 使用原图中的 " $P_{os+g}$ " 部分，显示橙色条 (Gradients) 也被切分。
- 显示显存从 31.4GB 降到 16.6GB 的对比。
- 可以加入一个示意图，展示梯度在反向传播过程中如何 Reduce-Scatter。

---

## Slide 9: ZeRO Stage 3 (FSDP): 参数分片 ( $P_{os+g+p}$ )

标题：ZeRO Stage 3 (FSDP): 极致瘦身，连“课本”也分片

核心内容：

ZeRO Stage 3 (简称  $P_{os+g+p}$ )，也被称为 **FSDP (Fully Sharded Data Parallel)**，在 Stage 2 的基础上，进一步分片**模型参数 (Parameters)**。

- 策略：
  - 优化器状态 和 梯度 都已分片。
  - 模型参数 也被分片：每张 GPU 只存储模型参数的一部分。

- 工作流 (关键):
  1. 前向传播 (Forward):
    - 当需要计算某一层时, 通过 **All-Gather** 从所有 GPU 收集该层的完整参数。
    - 计算完成后, **立即释放**完整的参数副本, 只保留本地的分片。
  2. 反向传播 (Backward):
    - 再次通过 **All-Gather** 收集该层的完整参数, 计算梯度。
    - 计算完成后, 通过 **Reduce-Scatter** 汇总并分发梯度。
    - **立即释放**完整的参数副本。
- 显存占用计算 (示例):
  - 总显存 =  $(2 \Psi / N_d + 2 \Psi / N_d + K * \Psi / N_d) = 1.9 \text{ GB}$
- 效果:
  - 显存占用达到极致, 与 GPU 数量呈线性反比关系。
  - 可以训练比单卡显存大几十倍的模型。
- 比喻: 每个人只有几页书, 要读整本书时, 大家把各自的几页凑起来, 读完立马拆散。

#### 视觉元素建议:

- 使用原图中的 " $P_{os+g+p}$ " 部分, 显示蓝色条 (Parameters) 也被切分。
- 显示显存从 16.6GB 降到 1.9GB 的对比。
- 关键图示: 使用原图中的 FSDP 流程图, 详细解释 All-Gather, Forward, Free, Backward, Reduce-Scatter 的循环。
- 突出显示 "Communication cost - 2 all gather (#param), 1 reduce-scatter (#param)"。

## Slide 10: 总结与实践建议

标题: 如何选择 ZeRO 阶段?

核心内容:

选择合适的 ZeRO 阶段是平衡显存、计算效率和通信开销的关键。

- 显存充足时 (小模型):
  - **DDP (传统数据并行)**: 最简单, 通信开销最低。
- 显存略有压力时 (中等模型):
  - **ZeRO Stage 1 ( $P_{os}$ )**: 性价比最高, 显存节省显著, 通信开销几乎不变。
- 显存紧张时 (大模型):
  - **ZeRO Stage 2 ( $P_{os+g}$ )**: 进一步节省显存, 通信开销可接受, 是许多大型模型训练的常用配置。
- 显存极度紧张时 (超大模型):

- **ZeRO Stage 3 (FSDP /  $P_{os+g+p}$ ):** 极致显存节省，但通信开销最大。适用于单卡无法容纳模型参数的情况。
  - **考虑因素：** 网络带宽 (NVLink, InfiniBand) 对 Stage 3 性能影响巨大。
  - **CPU Offload：** 可选功能，将部分数据卸载到 CPU 内存，进一步突破显存限制。

#### 视觉元素建议：

- 一个决策树或流程图，根据模型大小和可用显存推荐不同的 ZeRO 阶段。
  - 可以添加不同阶段的显存占用和通信开销的对比柱状图。
- 

## Slide 11: 总结与实践建议 (续)

标题：ZeRO/FSDP 的优势与考量

#### 优势：

- **突破显存限制：** 训练更大规模的模型，实现 SOTA 性能。
- **效率提升：** 减少 OOM 错误，提高训练稳定性。
- **易用性：** DeepSpeed 和 PyTorch FSDP 等框架提供了开箱即用的实现。

#### 考量：

- **通信开销：** 随着 ZeRO 阶段的提升，通信量增加，可能成为性能瓶颈。
- **网络带宽：** 高带宽、低延迟的网络对于 Stage 3 至关重要。
- **复杂性：** 相比 DDP，配置和调试可能更复杂。

#### 实践建议：

- **从小模型开始：** 逐步尝试不同 ZeRO 阶段，找到最佳平衡点。
- **监控资源：** 密切关注 GPU 显存使用率和网络带宽利用率。
- **硬件匹配：** 根据模型规模和网络条件选择合适的硬件配置。

#### 视觉元素建议：

- 一个天平，一边是“显存节省”，一边是“通信开销”。
  - 一些 DeepSpeed 或 PyTorch FSDP 的 Logo。
- 

## Slide 12: Q&A

标题：问答环节 (Q&A)



核心内容：

感谢您的聆听！现在是提问时间。

视觉元素建议：

- 一个大大的问号。
- 你的联系方式或团队 Logo。

希望这份详细的幻灯片内容能帮助你进行一次精彩的分享！

## Actual picture of how FSDP / ZeRO stage 3 works

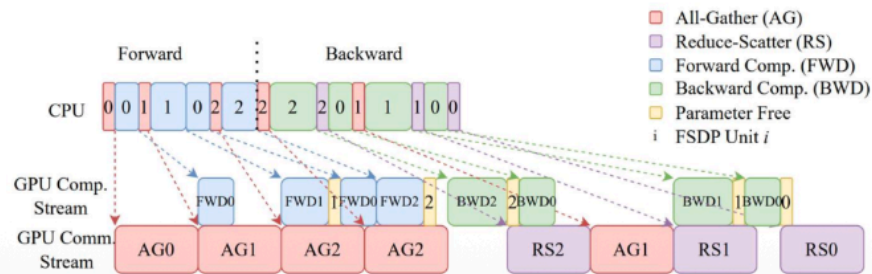
Let's walk through a FSDP example to see some important ideas

### Incremental computation / communication

- Parameters / gradients are requested / sent and then immediately freed

### Overlapping communication and computation ( $(W_1W_0 + W_2W_0)x = y$ )

- The all-gathers happen all at once while forward happens, masking the comm cost.



<https://arxiv.org/pdf/2304.11277.pdf>

这张图非常硬核，它展示了 **FSDP (ZeRO Stage 3)** 在微观时间轴上究竟是如何执行的。

它主要解释了两个核心问题：“怎么省显存”和“怎么不让速度变慢”。

我们可以把这张图分成三个部分来解读：

## 1. 核心理念 (顶部的文字)

- **增量计算 (Incremental computation):**
  - 这就是“省显存”的秘诀。
  - **机制：**参数/梯度是随用随取的。一旦计算完成，立刻释放 (Immediately freed)。
  - 对应图中：那些黄色的细条块 (Parameter Free)。
- **通信与计算重叠 (Overlapping):**
  - 这就是“保速度”的秘诀。
  - **机制：**在 GPU 计算当前层的同时，网络已经在后台悄悄下载下一层的参数了。

- **目的：**掩盖通信成本 (Masking the comm cost)。只要计算的时间比下载的时间长，你就感觉不到下载的延迟。
- 

## 2. 图解流水线 (中间的色块条)

这张图把时间轴分成了三条“跑道”，从左向右进行：

### 第一条跑道：CPU (指挥官)

- CPU 负责发号施令。你可以看到 CPU 的色块总是比下面的 GPU 快一点点，因为它要提前安排任务 (Launch Kernels)。
- 数字 0, 1, 2 代表第几层 (Layer/Unit)。

### 第二条跑道：GPU Comp. Stream (计算流水线 - 干活的)

这是 GPU 核心进行数学运算的地方。

- **FWD (蓝色):** 前向传播计算。FWD0 算第0层，FWD1 算第1层...
- **BWD (绿色):** 反向传播计算。顺序是反的，先算最后一层 BWD2，再算 BWD1 ...
- **黄色细条 (Parameter Free):** 这是重点！
  - 注意看每个 FWD 或 BWD 色块结束的瞬间，都有一个黄色小条。
  - 这意味着：“这一层算完了？好，马上把刚才借来的参数删掉！”这就是为什么 FSDP 极度节省显存。

### 第三条跑道：GPU Comm. Stream (通信流水线 - 搬运工)

这是网卡 (NVLink/InfiniBand) 在疯狂传输数据。

- **AG (红色 - All-Gather):** “借参数”。把这一层的完整参数从其他卡上拉取过来。
  - **RS (紫色 - Reduce-Scatter):** “还梯度”。把算好的梯度同步并切分给其他人。
- 

## 3. 这里的“魔法”在哪里？(看垂直对应关系)

请注意看图中虚线连接的对应关系，这就是 **Overlap (重叠)** 的精髓：

### 1. 前向传播时的“预取” (Prefetching):

- 看 **FWD0** (计算第0层) 的时候，正下方的通信流在做什么？
- 它在做 **AG1** (下载第1层的参数)！

- **含义**：当 GPU 还在切第一道菜 (Layer 0) 的时候，帮厨已经在冰箱里拿第二道菜的食材 (Layer 1) 了。
- **结果**：当 FWD0 算完时，AG1 刚好下载好，GPU 可以无缝衔接开始算 FWD1，完全不用停下来等数据。

## 2. 反向传播时的“双向奔赴”：

- 看 **BWD2** (算第2层的梯度) 后面。
- 一旦梯度算好，通信流立刻开始 **RS2** (把第2层的梯度发出去同步)。
- 同时，通信流还在做 **AG1** (为了算下一层 BWD1，提前把 Layer 1 的参数拉回来)。

## 总结

这张图告诉你，FSDP 不仅仅是把模型切碎那么简单，它是一场精密的配合战：

1. **显存利用极致**：用完立刻删 (黄色条)，绝不多留一毫秒。
2. **时间管理大师**：永远在计算当前层的同时，去下载下一层的数据 (AG块藏在FWD块下面)。

**潜在风险**：如果你的网速太慢，导致 **AG (红色块)** 比 **FWD (蓝色块)** 还要长，那么 GPU 就不得不停下来等待数据，训练速度就会变慢。这就是为什么大模型训练对网络带宽要求极高。

### What's the point?

Distributed data parallel costs  $2 * \# \text{ param communication}$

#### What about ZeRO?

- **Zero stage 1** is  $2 * \# \text{ param}$  – it's free! – you might as well always do it
- **Zero stage 2** is  $2 * \# \text{ param}$  – it's (almost) free (ignoring overhead)
- **Zero stage 3** is  $3 * \# \text{ param}$  – 1.5x comm cost, but that's not bad! (ignoring latency..)

This is also conceptually very simple – write a FSDP block wrapper.

这张图的核心目的是对比不同分布式训练策略的通信成本 (**Communication Cost**)，并以此论证为什么 ZeRO (尤其是 Stage 1 和 2) 是极具性价比的选择，以及为什么 Stage 3 也是可以接受的。

下面我详细解释图中的每一个要点：

## 1. 基准线：Distributed Data Parallel (DDP)

原文: "Distributed data parallel costs  $2 \times \# \text{ param}$  communication"

- 含义: 在传统的分布式数据并行 (DDP) 中, 通信量是模型参数数量的 **2倍**。
  - 为什么是 2 倍?
    - DDP 的核心操作是 **All-Reduce** (全归约), 用于同步所有 GPU 上的梯度。
    - 在环形 All-Reduce 算法中, 每个节点需要发送一次数据, 接收一次数据, 或者更准确地说, All-Reduce 操作在数学上等价于一次 Reduce-Scatter (1倍) + 一次 All-Gather (1倍)。
    - 所以, 总通信量 = **2 \* 模型参数量**。
- 

## 2. ZeRO Stage 1: 免费的午餐

原文: "Zero stage 1 is  $2 \times \# \text{ param}$  – it's free! – you might as well always do it"

- 含义: ZeRO Stage 1 的通信量也是 **2倍**, 和 DDP 完全一样。
  - 为什么是“免费”的?
    - Stage 1 只切分了优化器状态 (**Optimizer States**)。
    - 在反向传播结束后, 各 GPU 仍然需要进行一次 **All-Reduce** 来同步梯度 (和 DDP 一样)。
    - 同步完梯度后, 各 GPU 只更新自己负责的那部分优化器状态和参数, 然后再通过 **All-Gather** 把更新后的参数同步给所有人 (或者在下一次 All-Reduce 中隐式完成)。
    - 结论: 既然通信量没变, 但显存却节省了 (省去了冗余的优化器状态), 那这就是**纯赚**的。所以作者建议“**you might as well always do it**” (你不如干脆一直开着它)。
- 

## 3. ZeRO Stage 2: 几乎免费

原文: "Zero stage 2 is  $2 \times \# \text{ param}$  – it's (almost) free (ignoring overhead)"

- 含义: ZeRO Stage 2 的通信量理论上也是 **2倍**。
- 机制:
  - Stage 2 切分了**梯度 (Gradients)**。
  - 它不再使用 All-Reduce。
  - 步骤 1: 梯度计算完后, 使用 **Reduce-Scatter** (1倍通信) 把梯度归约到对应的 GPU 上。
  - 步骤 2: 各 GPU 更新参数。

- 步骤 3: 使用 **All-Gather** (1倍通信) 把更新后的参数广播给所有人。
  - 总计: 1 (Reduce-Scatter) + 1 (All-Gather) = **2倍**。
  - 为什么说 "(almost) free"?
    - 虽然总通信量还是 2 倍, 但因为操作被拆分成了两步 (Reduce-Scatter 和 All-Gather), 可能会引入一些细微的**开销 (Overhead)** 或延迟, 不像 DDP 的 All-Reduce 那么高度优化和连续。但总体上, 它依然保持了和 DDP 相同的通信量级, 同时大幅节省了显存 (梯度也不用存全量的了)。
- 

## 4. ZeRO Stage 3: 代价可控的终极方案

原文: "Zero stage 3 is 3\*# param – 1.5x comm cost, but that's not bad! (ignoring latency..)"

- 含义: ZeRO Stage 3 (FSDP) 的通信量是 **3倍**, 也就是 DDP 的 **1.5倍**。
  - 为什么是 3 倍?
    - Stage 3 切分了**模型参数 (Parameters)**。每次计算都需要去借参数。
    - **前向传播 (Forward)**: 需要一次 **All-Gather** 来收集参数。 (**1倍**)
    - **反向传播 (Backward)**:
      - 再次需要一次 **All-Gather** 来收集参数以计算梯度。 (**1倍**)
      - 计算完梯度后, 需要一次 **Reduce-Scatter** 来同步和切分梯度。 (**1倍**)
    - 总计: 1 + 1 + 1 = **3倍**。
  - 评价:
    - 虽然通信量增加了 50% (1.5x), 但这换来的是**显存占用的线性下降** (可以训练比原来大几十倍的模型)。
    - "**ignoring latency**" (**忽略延迟**): 这是一个重要的注脚。虽然总数据量是 1.5 倍, 但因为通信被拆得很碎 (每层都要通信), 如果网络延迟高 (**Latency**), 实际变慢的程度可能会远超 1.5 倍。但在高带宽网络下, 这个代价是完全可以接受的。
- 

## 5. 总结 (What's the point?)

这张图的逻辑链条是:

1. **DDP 是基准**, 通信量是 2。
2. **Stage 1 & 2** 既能省显存, 通信量又没变 (还是 2), 所以**性价比极高**, 几乎是无脑选。
3. **Stage 3** 虽然通信量增加了 50% (变成了 3), 但考虑到它带来的巨大显存红利 (能训超大模型), 这个代价是**划算的**。

最后一句 "conceptually very simple - write a FSDP block wrapper" 是在鼓励开发者：虽然原理听起来复杂，但实现起来其实就是把模型层包一层 Wrapper（包装器），代码层面并不难。

## Beyond data parallel – model parallelism

### Scaling up in memory (without changing batch size) with model parallelism

#### What model parallelism is..

- It splits up the parameters across GPUs (like zero3)..
- But communicate activations (while zero3 sends params).

#### We cover two different types of parallelism

1. Pipeline parallel
2. Tensor parallel (+ Sequence parallel)

These correspond to two different ways of cutting up the model.

这张幻灯片标志着从 **数据并行 (Data Parallelism)** 到 **模型并行 (Model Parallelism)** 的主题转换。

它主要解释了什么是模型并行，以及它与之前讨论的 ZeRO/FSDP 有何不同。

## 1. 核心定义：什么是模型并行？

原文: "What model parallelism is... It splits up the parameters across GPUs (like zero3).. But communicate activations (while zero3 sends params)."

这是最关键的区分点：

- **共同点**: 模型并行和 ZeRO Stage 3 (FSDP) 一样，都会把**模型参数切分 (Split)** 到不同的 GPU 上。单张卡都存不下完整的模型。
- **不同点 (通信内容)**:
  - **ZeRO Stage 3 (FSDP)**: 通信的是 **参数 (Params)**。
    - 逻辑: "我要算这一层了，把这一层的完整参数发给我！" -> 拿到完整参数 -> 本地计算 -> 扔掉参数。
  - **模型并行 (Model Parallelism)**: 通信的是 **激活值 (Activations)**（即中间计算结果）。
    - 逻辑: "我只负责算这一层的一部分（或者这一层），算完的结果（激活值）发给下一个 GPU 继续算。" -> 参数永远不动，只传数据。

## 2. 为什么要用模型并行？

原文: "Scaling up in memory (without changing batch size)"

- 突破单卡显存限制: 当模型大到连 ZeRO Stage 3 都搞不定, 或者需要在不增加 Batch Size 的情况下训练超大模型时, 模型并行是必须的。
- 不仅仅是显存: 有时候为了降低通信延迟或适应特定的硬件拓扑, 模型并行也是一种选择。

### 3. 模型并行的两种主要类型

原文: "We cover two different types of parallelism... These correspond to two different ways of cutting up the model."

幻灯片列出了两种“切蛋糕”的方式:

#### 1. 流水线并行 (Pipeline Parallelism - PP)

- 切法: 横着切 (按层切)。
- 比喻: 汽车装配线。
  - GPU 1 负责第 1-10 层。
  - GPU 2 负责第 11-20 层。
  - ...
- workflow: 数据从 GPU 1 进去, 算完把结果 (激活值) 传给 GPU 2, GPU 2 接着算。
- 特点: 减少了单卡的参数量, 但容易出现“气泡” (某个 GPU 在等别人的结果, 处于空闲状态)。

#### 2. 张量并行 (Tensor Parallelism - TP)

- 切法: 竖着切 (按矩阵切)。
- 比喻: 多人合力搬一块大石头。
  - 一个巨大的矩阵乘法  $A \times B$ 。
  - GPU 1 负责算左半边。
  - GPU 2 负责算右半边。
- workflow: 每一层的计算都需要所有 GPU 参与, 算完后需要立刻通信合并结果。
- 特点: 通信极其频繁 (每层都要通信), 所以通常只在同一台机器内部 (NVLink 连接) 使用。
- **Sequence Parallelism (序列并行)**: 这是 TP 的一种变体或补充, 针对长序列数据 (如 Transformer 中的 Sequence Length) 进行切分, 进一步节省显存。

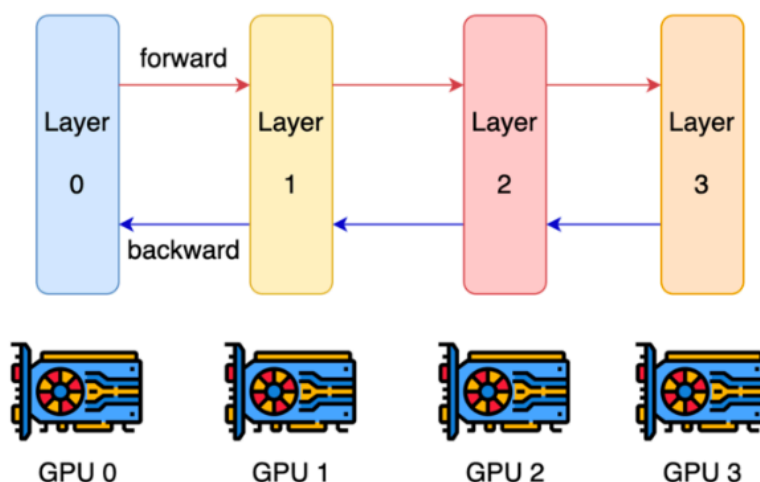
### 总结

这张图在告诉听众: 我们已经讲完了通过“借参数”来省显存的方法 (ZeRO), 现在我们要讲通过“切分计算任务”并传递中间结果来省显存的方法 (模型并行)。

- **ZeRO**: 每个人轮流拿完整的书读一页。

- **Pipeline Parallel:** 每个人撕下书的一章，读完把心得传给下一个人。
- **Tensor Parallel:** 每一页都被撕成两半，两个人并排坐着，每人读半行，拼起来理解。

## Layer-wise parallel



**Layer-wise parallel** cuts up layers, assigns some subset to GPUS.

**Activations and partial gradients** are passed back and forth

这张图展示的是 **Layer-wise Parallelism**（按层并行），这通常也被称为 **Pipeline Parallelism**（流水线并行）的最基础形式。

### 核心概念

#### 1. 切分方式 (Cutting up layers):

- 模型被“横着切”成了多段。
- 假设模型有 4 层 (Layer 0, 1, 2, 3)。
- **GPU 0** 独占 **Layer 0** 的参数。
- **GPU 1** 独占 **Layer 1** 的参数。
- 以此类推。
- 这意味着每张 GPU 不需要存储整个模型，只需要存储它负责的那几层的参数。这极大地降低了单卡的显存压力。

#### 2. 数据流向 (Passed back and forth):

- **Forward (前向传播 - 红色箭头):**
  - 数据 (Input) 进入 GPU 0，经过 Layer 0 计算。
  - 计算出的**激活值 (Activations)** 被发送给 GPU 1。
  - GPU 1 接收激活值，经过 Layer 1 计算，把结果发给 GPU 2...



- 直到 GPU 3 算出最终结果 (Loss)。
- **Backward (反向传播 - 蓝色箭头):**
  - GPU 3 根据 Loss 计算出 Layer 3 的梯度, 并将对输入的梯度 (**Partial Gradients**) 传回给 GPU 2。
  - GPU 2 接收梯度, 计算 Layer 2 的梯度, 再传回给 GPU 1...
  - 直到传回 GPU 0。

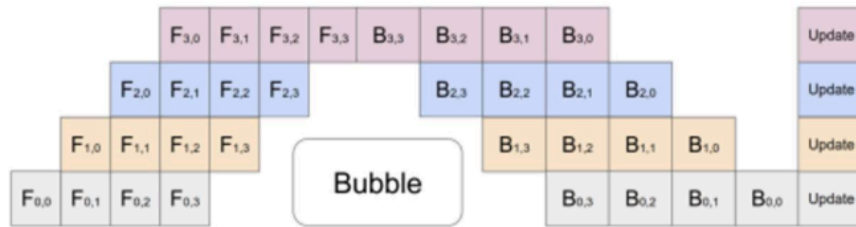
## 优缺点分析

- 优点:
  - **显存节省明显:** 模型参数被物理地分散了。如果模型有 100GB, 你有 4 张卡, 每张卡只存 25GB。
  - **通信量较小:** 只需要在 GPU 之间传递激活值和梯度 (通常比参数小得多), 而且只在相邻 GPU 之间通信 (点对点通信), 不需要像 DDP 那样做全员广播 (All-Reduce)。
- 缺点 (这张图暗示的问题):
  - **空闲等待 (Idle Time / Bubbles):**
    - 注意看图: 当 GPU 0 在算 Layer 0 时, GPU 1, 2, 3 都在干等 (因为数据还没传过来)。
    - 当数据传到 GPU 3 时, GPU 0, 1, 2 可能已经没事干了。
    - 这种“一人干活, 三人围观”的现象会导致 GPU 利用率极低。
  - 这就是为什么后来出现了 **GPipe** 或 **PipeDream** 等更高级的流水线并行技术, 它们通过把数据切成更小的 **Micro-batches** 来填补这些空闲时间, 让流水线“流动”起来。

## 总结

这张图是流水线并行的**逻辑示意图**。它直观地展示了“接力跑”的模式: GPU 0 跑第一棒, 传给 GPU 1 跑第二棒... 虽然省力 (省显存), 但如果配合不好, 等待交接棒的时间会很长。

## A solution: pipeline parallel



### Solution: Pipeline-parallel.

Process 'micro-batches' (in this case, 4).  
Send off the first microbatch and start computing the second.

The ratio of bubble time to useful compute is  $\dots \frac{n_{stages}-1}{n_{micro}}$  so we need a big batch size!

这张图展示了解决上一张图“空闲等待”问题的方案：基于微批次（Micro-batches）的流水线并行（Pipeline Parallelism），通常特指 GPipe 的调度模式。

## 1. 核心解决方案：微批次 (Micro-batches)

原文: "Process 'micro-batches' (in this case, 4). Send off the first microbatch and start computing the second."

- 以前的问题: 上一张图中，我们把一大坨数据（整个 Batch）一次性塞给 GPU 0，算完才给 GPU 1。导致 GPU 0 算的时候，GPU 1 闲着；GPU 1 算的时候，GPU 0 闲着。
- 现在的做法：
  - 把一个大的 Batch 切分成多个小的 **Micro-batches**（图中的例子切成了 4 份，编号 0, 1, 2, 3）。
  - 流水线作业：
    - GPU 0 算完第 1 个微批次 (F0,0)，立刻把它传给 GPU 1。
    - GPU 1 开始算第 1 个微批次 (F1,0)。
    - 关键点: 与此同时，GPU 0 不休息，立刻开始算第 2 个微批次 (F0,1)。
  - 这就形成了图中像“俄罗斯方块”一样的紧密堆叠结构，大大提高了 GPU 的利用率。

## 2. 图解符号含义

- 纵轴: 代表不同的 GPU (Stage)。最下面是 GPU 0，最上面是 GPU 3。
- 横轴: 代表时间。
- 方块:

- $F_{\{stage, microbatch\}}$  (蓝色/黄色): 前向传播。例如  $F_{0,0}$  是 GPU 0 处理第 0 个微批次。
- $B_{\{stage, microbatch\}}$  (紫色/深色): 反向传播。例如  $B_{3,3}$  是 GPU 3 处理第 3 个微批次的梯度。
- **Update:** 所有微批次的梯度都算完后，统一更新模型参数。

### 3. 无法消除的痛：气泡 (Bubble)

图中中间的大片空白: "Bubble"

尽管用了微批次，流水线并行依然存在**气泡**（即 GPU 空闲的时间）：

- **启动阶段 (Fill phase):** 在开始时，GPU 3 必须等数据从 GPU 0 -> 1 -> 2 一路传上来，这段时间 GPU 3 是闲着的。
- **收尾阶段 (Drain phase):** 在结束时，GPU 0 必须等梯度从 GPU 3 -> 2 -> 1 一路传回来，这段时间 GPU 0 是闲着的。
- 图中中间那个巨大的 Bubble 空白区域，以及左上角和右下角的空白，都是浪费掉的算力。

### 4. 数学公式与结论

公式: Ratio of bubble time  $\approx \frac{n_{stages}-1}{n_{micro}}$

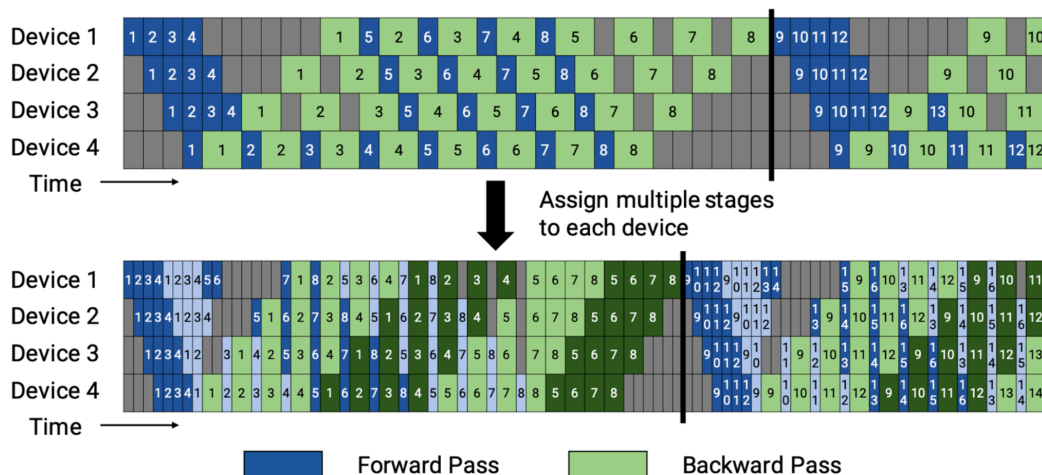
这个公式揭示了流水线并行的**效率法则**：

- **分子 ( $n_{stages} - 1$ ):** 你的流水线越长（GPU 越多），气泡就越大，效率越低。
- **分母 ( $n_{micro}$ ):** 你的微批次数量越多，气泡在总时间里的占比就越小，效率越高。

**最终结论 (So we need a big batch size!):**

为了让流水线并行高效运行，你必须把分母搞大。这意味着你需要一个**非常大的全局 Batch Size**，这样才能切分出足够多的 Micro-batches 来填满流水线，稀释掉气泡的影响。

# Trading communication bandwidth for utilization



Some more crazy pipeline patterns can improve utilization, but at the cost of bandwidth 这两页幻灯片展示了流水线并行 (Pipeline Parallelism) 技术的进阶演变：从为了提高利用率而牺牲带宽的复杂调度，到最新的 **Zero Bubble (零气泡)** 技术。

我将分两部分详细解释。

## 第一页：用带宽换利用率 (Trading communication bandwidth for utilization)

这一页的核心思想是：如果我们让每个 GPU 负责不连续的多个阶段 (Stages)，能不能把气泡填满？

### 1. 上半部分图：传统的 1F1B (One-Forward-One-Backward)

- **结构:** 4 个 Device，每个 Device 负责一段连续的模型层 (Device 1 负责 Stage 1，Device 2 负责 Stage 2...)。
- **问题:** 你可以看到明显的气泡 (灰色空白区域)。
  - 开始时，Device 4 必须等 Device 1->2->3 算完，前面全是空的。
  - 结束时，Device 1 必须等 Device 4->3->2 算完，后面全是空的。
  - 这种“三角形”的空闲区域是传统流水线的硬伤。

### 2. 下半部分图：交错式流水线 (Interleaved Pipeline)

这是 Megatron-LM 等框架中引入的高级技术。

- **做法:** "Assign multiple stages to each device".
  - 不再是 Device 1 只负责 Layer 1-10。

- 而是把模型切得更碎，比如 Device 1 负责 Layer 1-5 和 Layer 21-25。
- Device 2 负责 Layer 6-10 和 Layer 26-30。
- 效果:
  - 你看下面的图，那个巨大的“三角形”气泡变小了，变成了很多细小的锯齿状气泡。
  - 利用率 (Utilization) 明显提高了，深色区域（干活的时间）变多了。
- 代价 (Cost):
  - 带宽消耗增加: 因为 Device 1 现在负责两个不连续的阶段，数据要在 Device 1 -> 2 -> ... -> 1 之间来回穿梭。通信次数变多了，对网络带宽的要求更高了。

## ‘Zero bubble’ pipelining

### Split up backwards into two parts

1. Backpropagating activations (z,x)
2. Computing weight gradients (W)



Figure 2: 1F1B pipeline schedule.

The second part can be done whenever

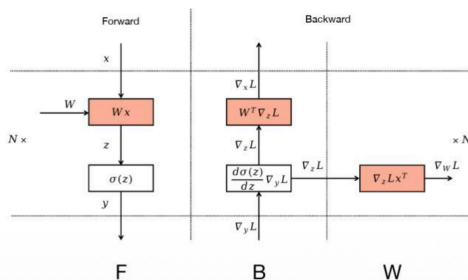


Figure 1: Computation Graph for MLP.

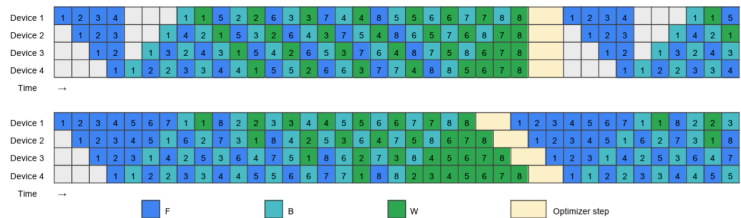


Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

## 第二页：零气泡流水线 ('Zero bubble' pipelining)

这一页介绍了一种更前沿的思路（来自论文 "Zero Bubble Pipeline Parallelism"），试图从数学原理上彻底消灭气泡。

### 1. 核心洞察：反向传播可以拆分

原文: "Split up backwards into two parts: 1. Backpropagating activations (B) ... 2. Computing weight gradients (W)"

这是整个技术的灵魂所在。传统的反向传播 (Backward) 被视为一个原子操作，但实际上它包含两个独立的数学步骤：

1. 算输入的梯度 (B): 为了传给上一层 (Previous Layer)，让上一层继续反向传播。这步必须紧接着做，否则流水线会卡住。

2. 算权重的梯度 (**W**): 为了更新当前层的参数。这步不着急! 只要在最终 Update 之前算完就行。

## 2. 图解计算图 (Figure 1)

- **F (Forward)**: 算出  $z$ 。
- **B (Backward Data)**: 算出  $\nabla_x L$  (传给上一层)。
- **W (Backward Weight)**: 算出  $\nabla_w L$  (存下来用于更新参数)。
- 关键点:  $W$  的计算不依赖于传给上一层的数据, 它是一个“旁路”。

## 3. 调度对比 (Figure 2 vs Figure 3)

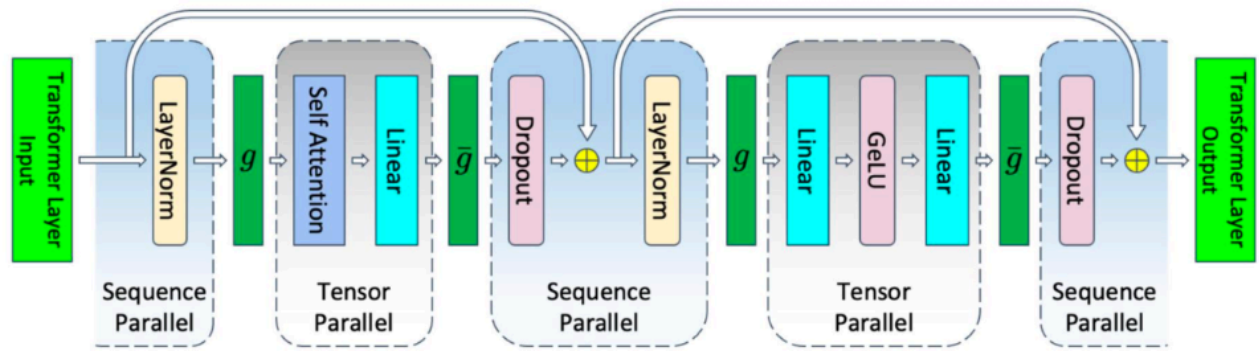
- **Figure 2: 传统的 1F1B 调度**
  - 你看那些深橙色的块 (Backward), 它们是连在一起的。
  - 结果就是开头和结尾有大片的浅黄色空白 (气泡)。
- **Figure 3: 手工设计的零气泡调度 (ZB-H1 / ZB-H2)**
  - 魔法操作: 把原本连在一起的深橙色块拆开了!
  - 蓝色 (**F**): 前向传播。
  - 深绿色 (**B**): 紧急的反向传播 (传给上一层)。
  - 浅绿色 (**W**): 不紧急的权重梯度计算。
  - 填空: 调度器把那些“不紧急”的 **W** 块, 塞到了原本是气泡的空白时间里!
  - 结果:
    - 注意看 Figure 3 的中间部分, 几乎被填得满满当当。
    - Device 1 在等待 Device 2 的数据时, 不再闲着, 而是去算之前攒下来的 **W** 任务。

## 总结

这两页展示了 AI 系统领域为了榨干 GPU 算力所做的极致优化:

1. **Interleaved Pipeline**: 通过把模型切得更碎, 让 GPU 轮流处理不同阶段的任务, 用通信换计算, 减小气泡。
2. **Zero Bubble**: 通过拆解反向传播的数学步骤, 把“不紧急”的计算任务 (算权重梯度) 挪到“等待时间”里去做, 从而实现理论上的零气泡。

# Making memory truly linear – sequence parallel



- Observation:** all the 10sbh terms are pointwise ops over the sequence  
... so split up the layer norm/dropout layers along the sequence axis.
- In the forward pass, ' $g$ ' is an all gather, ' $\bar{g}$ ' is reduce-scatter
  - In the backward pass, the two are reversed.

这张图展示了 **序列并行 (Sequence Parallelism)**，这通常是作为 **张量并行 (Tensor Parallelism)** 的一个重要补充优化，旨在进一步减少显存占用。

## 1. 核心问题：张量并行还不够完美

在标准的 Megatron-LM 风格的张量并行中：

- 矩阵乘法 (**Linear, Attention**) 被切分了（图中虚线框住的 Tensor Parallel 部分）。这意味着参数和计算都被分摊到了多个 GPU 上。
- 但是，像 LayerNorm 和 Dropout 这样的操作，通常是在完整的序列上进行的。这意味着在做这些操作时，每个 GPU 都需要存储完整的激活值 (**Activations**)。
- 对于超长序列（比如处理长文档或基因序列），这些激活值会占用巨大的显存，成为瓶颈。

## 2. 解决方案：序列并行 (Sequence Parallel)

**Observation:** "all the ... terms are pointwise ops over the sequence... so split up the layer norm/dropout layers along the sequence axis."

- 观察: LayerNorm 和 Dropout 主要是 **逐点运算 (Pointwise Ops)**。也就是说，第 1 个 token 的 LayerNorm 结果只取决于第 1 个 token（以及均值方差），跟第 100 个 token 没关系。
- 做法: 既然如此，我们为什么要在每个 GPU 上都存完整的序列呢？
  - 把序列切成  $N$  份 ( $N = \text{GPU 数量}$ )。
  - GPU 1 只负责存第 1 份序列的激活值，并计算它的 LayerNorm/Dropout。
  - GPU 2 只负责存第 2 份...
- 效果: 这样一来，LayerNorm 和 Dropout 阶段的显存占用也变成了  $1/N$ ，真正实现了**显存占用随 GPU 数量线性减少 (Making memory truly linear)**。

### 3. 数据流与通信操作 ( $g$ 和 $\bar{g}$ )

为了实现这种切换，需要在 Tensor Parallel 和 Sequence Parallel 之间进行转换：

- $g$  (**All-Gather**):
  - 位置: 在进入 Tensor Parallel 区域之前（例如进入 Self-Attention 之前）。
  - 作用: 因为 Self-Attention 需要看到所有的 token 才能计算注意力，所以必须把分散在各 GPU 上的序列片段拼凑成完整的序列。
  - 动作: 大家把各自手里的序列片段广播给所有人。
- $\bar{g}$  (**Reduce-Scatter**):
  - 位置: 在离开 Tensor Parallel 区域之后（例如 Linear 算完之后）。
  - 作用: Tensor Parallel 输出的是完整的序列结果（或者需要归约的结果），但为了进入 Sequence Parallel 节省显存，我们不需要每个人都存全量的结果。
  - 动作: 把结果归约（Reduce），然后每个人只拿走自己负责的那一小段（Scatter），丢掉剩下的。

### 4. 总结

这张图描述的是一个 Transformer 层内部的精细化切分：

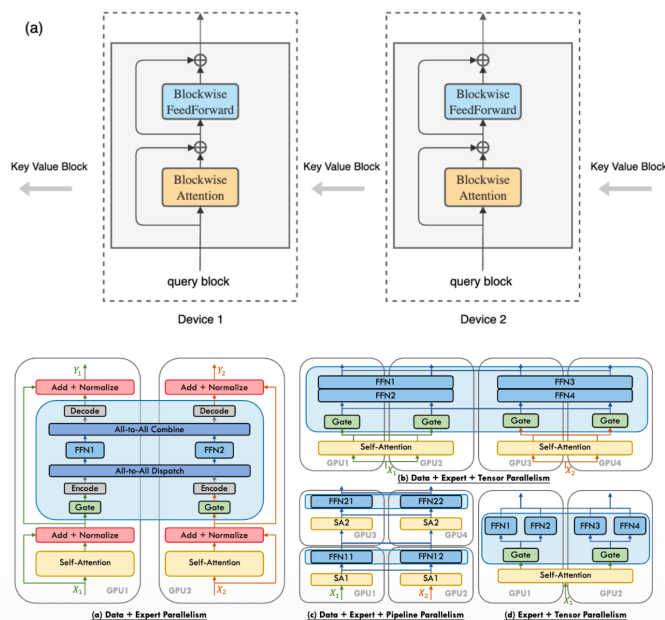
- 计算密集型部分 (**Linear/Attn**): 用 **Tensor Parallel**（切分权重）。
- 显存密集型/逐点运算部分 (**LayerNorm/Dropout**): 用 **Sequence Parallel**（切分数据序列）。
- 连接点: 用  $g$  (All-Gather) 和  $\bar{g}$  (Reduce-Scatter) 来在两种模式间无缝切换。

这种组合拳打法是目前训练超大模型（如 GPT-3, LLaMA 等）且序列很长时的标准配置。



## Other parallelism strategies

**Context parallel / Ring attention**  
(split activations across GPUs in a long sequence)



这张幻灯片介绍了除了前面提到的数据并行、流水线并行、张量并行之外，两种针对特定场景的高级并行策略。

### 1. Context Parallel / Ring Attention (上下文并行 / 环状注意力)

核心思想: "Split activations across GPUs in a long sequence" (在长序列中跨 GPU 切分激活值)。

这主要是为了解决 超长上下文 (Long Context) 带来的显存爆炸问题。

- **问题场景:** 当你训练一个支持 100k 甚至 1M token 上下文的模型时，仅仅存储 Attention 矩阵的中间结果 ( $Q \times K^T$ ) 就能把显存撑爆。普通的 Sequence Parallel 只能切分 LayerNorm/Dropout，对 Attention 内部的巨大矩阵无能为力。
- **解决方案 (Ring Attention):**
  - **切分 Query (Q):** 每个 GPU 只负责计算一部分 Query 的注意力。
  - **传递 Key/Value (K/V):**
    - Device 1 拿着自己的  $Q_1$ ，先和自己的  $K_1, V_1$  算。
    - 然后，Device 1 把  $K_1, V_1$  传给 Device 2，同时接收 Device 2 传来的  $K_2, V_2$  (或者反过来，接收上一个 Device 的 KV)。
    - Device 1 接着用  $Q_1$  和新收到的  $K_2, V_2$  算。
    - 这个过程像一个环 (Ring) 一样流转，直到每个  $Q$  都看过了所有的  $K, V$ 。
- **图示 (a):**
  - 你可以看到 "Key Value Block" 在 Device 1 和 Device 2 之间像传送带一样传递。
  - "Blockwise Attention" 表示注意力计算是分块进行的，不需要一次性把整个巨大的矩阵算出来。

## 2. Expert Parallel (专家并行)

核心思想: "Split experts across GPUs" (跨 GPU 切分专家)。

这是专门针对 **MoE (Mixture of Experts, 混合专家模型)** 架构的并行策略 (比如 GPT-4, Mixtral 8x7B)。

- **MoE 架构回顾:** 模型里有很多个“专家”网络 (FFN)，对于每个输入 token，只有少数几个专家会被激活 (Gating)。这意味着模型参数量巨大，但计算量相对较小。
- **并行策略:**
  - **Data Parallel (DP):** 复制所有专家到每个 GPU。太浪费显存。
  - **Expert Parallel (EP):**
    - **切分方式:** 把不同的专家放在不同的 GPU 上。比如 GPU 1 放专家 1-2, GPU 2 放专家 3-4。
    - **通信 (All-to-All):**
      - **Dispatch:** 当 token 进来后，如果 Gate 决定它需要专家 3 处理，这个 token 就会被发送到 GPU 2。
      - **Combine:** 专家 3 算完后，结果再被发回原来的 GPU。
- **图示 (下半部分):**
  - **(a) Data + Expert Parallelism:** 展示了基本的 EP 流程。输入数据 ( $X_1, X_2$ ) 经过 Gate 后，通过 **All-to-All Dispatch** 被分发给不同的 FFN (专家)，算完再 **All-to-All Combine** 回来。
  - **(b), (c), (d):** 展示了 EP 如何与其他并行策略 (Tensor Parallel, Pipeline Parallel) 混合使用，形成复杂的 3D/4D 并行策略。

## 总结对比

| 策略                           | 解决的核心痛点                    | 关键操作                             | 适用场景                       |
|------------------------------|----------------------------|----------------------------------|----------------------------|
| Context Parallel / Ring Attn | 序列太长 (显存不够存 Attention Map) | KV Block 在 GPU 间轮转 (Ring)        | 训练长文本模型 (如 128k+ context)  |
| Expert Parallel              | 模型参数太大 (MoE 架构)            | Token 根据路由分发给不同 GPU (All-to-All) | 训练稀疏大模型 (如 Mixtral, GPT-4) |

这张图展示了现代大模型训练已经进入了**精细化操作**阶段：不再是简单地切分模型或数据，而是根据具体的算子特性 (Attention 的二次方复杂度) 或模型架构特性 (MoE 的稀疏性) 来设计定制化的并行方案。

没错，你的理解非常精准。**Router (或者叫 Gate)** 是跑在每一张 GPU 上的。

并没有一个物理上独立的“路由器层”或者“路由器 GPU”来专门负责分发数据，那样会成为巨大的单点瓶颈。

以下是 MoE（混合专家模型）中 Router 和 All-to-All 配合工作的具体流程，帮你彻底理清这个逻辑：

## 1. Router (Gate) 是什么？在哪里？

- **本质:** Router 其实就是一个很小的全连接层（Linear Layer）加上一个 Softmax。它的参数量相对于那些庞大的“专家（Experts）”来说九牛一毛。
- **位置:** 在训练开始时，这个 Router 的参数会被复制 to 每一张 GPU 上（就像数据并行 Data Parallel 一样）。
- **动作:**
  - 假设 GPU 1 拿到了一批数据（Batch A）。
  - GPU 1 会在本地运行这个 Router 层。
  - Router 会告诉 GPU 1：“你手里的 Token 1 需要去 GPU 2 找专家 X，Token 2 需要去 GPU 3 找专家 Y，Token 3 留在本地找专家 Z。”

## 2. 为什么要用 All-to-All？

当每张 GPU 都在本地算完了“谁该去哪”之后，数据物理位置和逻辑位置就不匹配了。这时候就需要 All-to-All 通信原语登场了。

- **Dispatch (分发) 阶段:**
  - 大家根据刚才 Router 的指示，同时交换数据。
  - GPU 1 把属于 GPU 2 的数据发过去。
  - GPU 2 把属于 GPU 1 的数据发过来。
  - **结果:** 交换完后，GPU 1 手里拿的都是“专家 1”需要处理的数据（不管这些数据原本是谁的）。
- **Compute (计算) 阶段:**
  - GPU 1 运行“专家 1”的网络层。
- **Combine (聚合) 阶段:**
  - 再次进行 All-to-All。
  - 把算好的结果发回给数据原本的拥有者。

## 3. 形象的比喻

想象有 4 个分公司（GPU），每个分公司都有自己的员工（Token）。

- **没有单独的邮局 (No Separate Router Layer):** 并没有一个总局把所有信件收上来再分发。
- **Router 在本地:** 每个分公司的前台（Router）自己看一眼本公司的信件，标好“这封信要去分公司 B”，“那封信要去分公司 C”。

- **All-to-All**: 每天固定时间，4 个分公司的卡车互相对开，直接把信件交换到目的地。

## 总结

**Router** 是逻辑判断，**All-to-All** 是物理运输。

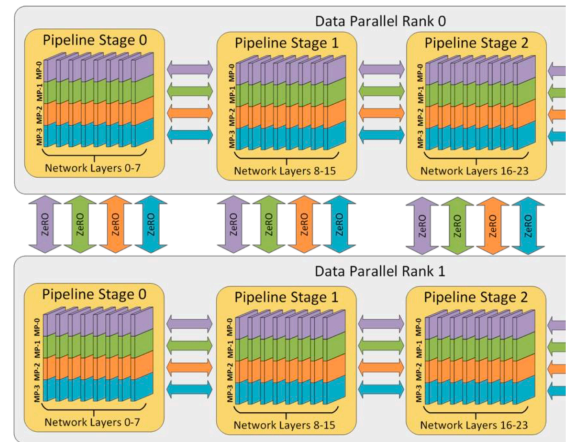
Router 代码在每张卡上都跑一遍，各自决定自己手里的数据去哪，然后大家通过 All-to-All 一起完成数据的“大挪移”。

## ‘3D parallelism’ – putting it all together

### Simple rules of thumb from the literature.

1. Until your model fits in memory..
  - Tensor parallel up to GPUs / machine
  - Pipeline parallel across machines
  - › (Or use Zero-3, depending on BW)
2. Then until you run out of GPUs
  - Scale the rest of the way with data parallel

If your batch size is small.. gradient accumulate to trade higher batch sizes for better communication efficiency.



这张图是整个并行训练策略的集大成者，总结了如何将数据并行 (Data Parallel)、张量并行 (Tensor Parallel) 和流水线并行 (Pipeline Parallel) 结合起来，形成所谓的 **3D 并行 (3D Parallelism)**。

它给出了一套非常实用的经验法则 (**Rules of Thumb**)，指导我们在实际训练大模型时如何配置这些策略。

## 1. 核心目标：把模型塞进显存，然后跑得快

训练大模型面临两个主要约束：

1. **显存 (Memory)**: 单张卡放不下模型。
2. **算力 (Compute)**: 单张卡跑得太慢。

## 2. 经验法则 (Rules of Thumb)

### 第一步：解决显存问题 (Until your model fits in memory..)

如果你的模型太大，单卡放不下，你需要把模型切开。切分的优先级如下：

1. **Tensor Parallel (TP) up to GPUs / machine**:
  - 优先用 **TP**: 首先尝试在单机内部 (intra-node) 使用张量并行。

- **原因:** TP 需要极高频的通信（每一层都要通信），通信量巨大。单机内部有 NVLink，带宽极高（600GB/s - 900GB/s），能扛得住。跨机通信走以太网或 InfiniBand，延迟和带宽都不如 NVLink，跑 TP 会慢死。
- **操作:** 如果你一台机器有 8 张卡，TP size 最多设为 8。

## 2. Pipeline Parallel (PP) across machines:

- **如果 TP 撑满了单机还放不下:** 这时候就需要跨机了。
- **使用 PP:** 流水线并行只需要在 Stage 之间传数据，通信频率低（只在切分点传），通信量相对较小。所以它适合**跨机**（inter-node）扩展。
- **操作:** 把模型切成几段，机器 A 跑第一段，机器 B 跑第二段...

## 3. Alternative: ZeRO-3:

- 如果你不想搞复杂的 TP/PP，也可以用 DeepSpeed 的 ZeRO-3（数据并行的一种变体，把参数切分到所有 GPU 上）。但这取决于你的网络带宽（BW），因为 ZeRO-3 通信量也很大。

## 第二步：解决速度问题 (Then until you run out of GPUs)

当你的模型已经能被一组 GPU（比如 4 张或 8 张）放下了，剩下的 GPU 怎么办？

- **Scale with Data Parallel (DP):**
  - **复制:** 把刚才那组“能放下模型的 GPU 组合”作为一个整体，复制多份。
  - **并行:** 让这些组合并行处理不同的数据（Batch），然后同步梯度。
  - **原因:** DP 是最容易扩展的，也是利用额外算力最直接的方式。

## 3. 图解 3D 并行结构

右边的图形象地展示了这三种并行是如何嵌套的：

- **最内层 (Tensor Parallel):**
  - 看那些彩色的切片（MP-0 到 MP-3）。
  - 这代表一个 Layer 被切分到了 4 张卡上（Model Parallelism Size = 4）。
  - 这通常发生在同一台机器内。
- **中间层 (Pipeline Parallel):**
  - 看横向的排列（Pipeline Stage 0 -> 1 -> 2）。
  - Stage 0 处理 Layer 0-7，Stage 1 处理 Layer 8-15...
  - 这代表模型在深度上被切分了。这通常跨越了不同的机器。
- **最外层 (Data Parallel):**
  - 看纵向的排列（Data Parallel Rank 0 和 Rank 1）。
  - 上面一整套流水线（Rank 0）和下面一整套流水线（Rank 1）是完全一样的副本。
  - 它们处理不同的数据，最后通过 **ZeRO** (或 AllReduce) 来同步梯度。

## 4. 关于 Batch Size 的补充

"If your batch size is small.. gradient accumulate..."

- **问题:** 流水线并行需要较大的 Batch Size 才能填满气泡（还记得之前的 Micro-batches 吗？）。
- **对策:** 如果你的全局 Batch Size 不够大，流水线效率会很低。这时候可以用 **梯度累积 (Gradient Accumulation)**:
  - 不急着更新参数。
  - 多跑几个小 Batch，把梯度攒起来。
  - 攒够了再更新。
  - 这样可以模拟出一个大的 Batch Size，提高流水线效率和通信效率。

## 总结：最佳实践配置

假设你有 1000 张卡，要训一个巨大的模型：

1. **先看单机:** 一台机器 8 张卡，先开 **TP=8**（把模型宽度切开，塞满单机显存）。
2. **再看跨机:** 如果模型深度还需要切，开 **PP=X**（比如 PP=4，跨 4 台机器）。
3. **最后扩容:** 剩下的卡全部用来做 **DP**（复制这 4 台机器的组合，并行吃数据）。

这就是目前工业界训练万亿参数模型的标准“三板斧”。